

LOM MATH

Deep dive in the magical world of LOM formulas

By Yuko (EUVN-154)

v1.2

Contents

History	3
Stats Calculation	4
Damage and Battle Calculation	5
General Notes	5
Player	5
Basic Attack	5
Combo.....	5
Counter	5
Critical damage	5
Evasion	5
Launch.....	6
Stun.....	6
Pierce	6
Block.....	6
Pal Inspire.....	6
Pal RES.....	6
Regeneration.....	7
Pals.....	8
Basic Attack	8
Combo.....	8
Critical Damage	8
Evasion	8
Launch.....	8
Pal Attributes	8
Skills	9
Skill Damage.....	9
Skill Critical Damage.....	9
Skills with Other Damage Types	9
Skills with Bleed Damage	10
Boss.....	10
Sailing Event.....	11
Cannons	11
Cannon Damage (Draft)	12
Cannon Basic Attack.....	12
Cannon Combo.....	12
HP-Based Damage.....	13
PvP Damage Reduction	15
Attributes Maximum Limits	19
Nerds Corner.....	20
Damage Resistance	20
Basic Attack.....	20
Combo.....	20
Counter	21
Skills	21
Evasion/Ignore Crit	21
Stun.....	22
Launch.....	22
Pierce/Block	22

Pal Inspire/Pal RES	22
HP-Based Damage.....	23
PvP Damage Reduction	24
Sailing Event.....	24
How to Browse the Code	25

History

Version	Description
<i>v1.0</i>	First version
<i>v1.1</i>	Add stun duration and control duration reduction Improve HP-based damage part
<i>v1.2</i>	Update sailing part Add description of new bonuses (pierce, block, pal inspire, pal RES, ignore crit, ignore regeneration) Add tutorial to browse the code Add attributes maximum cap

Stats Calculation

Some stats are composed of 3 parts: an initial value (orange in the picture), a “base” or “bonus” value (red in the picture) and a global value (green in the picture). For example:

Your Team	
1605.24% 	
Attribute Source	Your Team
Crit RES Total	100%
Initial Attributes	100%
Crit RES Bonus Total	1428.8%
Mount	40%
Soul	920.5%
Awaken Seal	148.3%
Back Accessory	320%
Global Crit RES Total	5%
Enchant	5%

The final value of these stats is calculated as follows:

$$Final\ HP = HP \times \left(1 + \frac{HP_{Base}}{100}\right) \times \left(1 + \frac{HP_{Global}}{100}\right)$$

$$Final\ ATK = ATK \times \left(1 + \frac{ATK_{Base}}{100}\right) \times \left(1 + \frac{ATK_{Global}}{100}\right)$$

$$Final\ DEF = DEF \times \left(1 + \frac{DEF_{Base}}{100}\right) \times \left(1 + \frac{DEF_{Global}}{100}\right)$$

$$Final\ Combo\ Multiplier = Combo\ Multiplier \times \left(1 + \frac{Combo\ DMG}{100}\right) \times \left(1 + \frac{Global\ Combo\ DMG}{100}\right)$$

$$Final\ Counter\ Multiplier = Counter\ Multiplier \times \left(1 + \frac{Counter\ DMG}{100}\right) \times \left(1 + \frac{Global\ Counter\ DMG}{100}\right)$$

$$Final\ Basic\ ATK\ Multiplier = Basic\ ATK\ Multiplier \times \left(1 + \frac{Basic\ ATK\ DMG}{100}\right) \times \left(1 + \frac{Global\ Basic\ ATK\ DMG}{100}\right)$$

$$Final\ Crit\ DMG = Crit\ DMG \times \left(1 + \frac{Crit\ DMG\ Bonus}{100}\right) \times \left(1 + \frac{Global\ Crit\ DMG}{100}\right)$$

$$Final\ Crit\ RES = Crit\ RES \times \left(1 + \frac{Crit\ RES\ Bonus}{100}\right) \times \left(1 + \frac{Global\ Crit\ RES}{100}\right)$$

Example 1

- ATK: 10000
- Base ATK: 200000%
- Global ATK: 300%
- $Final\ ATK = 1 + 10000 \times \left(1 + \frac{200000\%}{100}\right) \times \left(1 + \frac{300\%}{100}\right) = 80040001 \cong 80.04M$

Example 2

- Combo Multiplier: 100%
- Combo DMG: 5000%
- Global combo DMG: 300%
- $Final\ Combo\ Multiplier = 100\% \times \left(1 + \frac{5000\%}{100}\right) \times \left(1 + \frac{300\%}{100}\right) = 20400\%$

All the other stats are simply the addition of all the parts.

Damage and Battle Calculation

General Notes

- If $(Player_{ATK} - Enemy_{DEF})$ is lower than 1, it'll be set to 1. In PvE, $Enemy_{DEF}$ is always 0.
- If $Crit RES$ is lower than 50%, it'll be set to 50%. In PvE, monsters might have some $Crit RES$.
- If $\frac{Crit DMG}{Crit RES}$ is lower than 1.5, it'll be set to 1.5. This means that the minimum critical damage is 150%.
- Damage resistances can become negative. For example, if your opponent has 15% $DMG RES$ and you use the *Smoke Bomb* skill, his $DMG RES$ will drop to -5%, meaning that the damage he takes will increase by 5%.
- The minimum damage that can be dealt is 1.
- In PvP, every damage is finally divided by the *PvP Damage Reduction Factor* (see PvP Damage Reduction).

Player

Basic Attack

Every time a player attacks, it performs a basic attack.

The damage of a basic attack depends on the *Basic ATK Multiplier* of the player and the *Basic ATK DMG RES* and $DMG RES$ of the enemy as follows:

$$Basic\ ATK\ DMG = (Player_{ATK} - Enemy_{DEF}) \times \frac{Player_{Basic\ ATK\ MULT}}{100} \times \left(1 - \frac{Enemy_{Basic\ ATK\ DMG\ RES}}{100}\right) \times \left(1 - \frac{Enemy_{DMG\ RES}}{100}\right)$$

Combo

A combo is triggered by a basic attack and its damage is calculated separately.

The chance of performing a combo depends on the *Combo* of the player and the *Ignore Combo* of the enemy as follows:

$$Combo\ \% = Player_{Combo\ \%} - Enemy_{Ignore\ Combo\ \%}$$

If $(Player_{Combo\ \%} - Enemy_{Ignore\ Combo\ \%})$ is lower than 0, it'll be set to 0%.

The damage of a combo depends on the *Combo Multiplier* of the player and the *Combo DMG RES* and $DMG RES$ of the enemy as follows:

$$Combo\ DMG = (Player_{ATK} - Enemy_{DEF}) \times \frac{Player_{Combo\ MULT}}{100} \times \left(1 - \frac{Enemy_{Combo\ DMG\ RES}}{100}\right) \times \left(1 - \frac{Enemy_{DMG\ RES}}{100}\right)$$

Counter

A counter is triggered when the player gets attacked.

The chance of performing a counter depends on the *Counter* of the player and the *Ignore Counter* of the enemy as follows:

$$Counter\ \% = Player_{Counter\ \%} - Enemy_{Ignore\ Counter\ \%}$$

If $(Player_{Counter\ \%} - Enemy_{Ignore\ Counter\ \%})$ is lower than 0, it'll be set to 0%.

The damage of a counter depends on the *Counter Multiplier* of the player and the *Counter DMG RES* and $DMG RES$ of the enemy as follows:

$$Counter\ DMG = (Player_{ATK} - Enemy_{DEF}) \times \frac{Player_{Counter\ MULT}}{100} \times \left(1 - \frac{Enemy_{Counter\ DMG\ RES}}{100}\right) \times \left(1 - \frac{Enemy_{DMG\ RES}}{100}\right)$$

Critical damage

Basic attack, combo and counter can be critical.

The chance of performing a critical attack depends on the *Crit Rate* of the player and the *Ignore Crit* of the enemy as follows:

$$Crit\ \% = Player_{Crit\ \%} - Enemy_{Ignore\ Crit\ \%}$$

In case of a critical attack, the damage will be multiplied by the ratio between the *Crit DMG* of the player and *Crit RES* of the enemy as follows:

$$Crit\ DMG = DMG \times \frac{Player_{Crit\ DMG}}{Enemy_{Crit\ RES}}$$

Evasion

When a player attacks, the attack can be evaded by the enemy depending on the *Evasion* of the enemy and the *Ignore Evasion* of the player as follows:

$$Evasion\ \% = (Enemy_{Evasion\ \%} - Player_{Ignore\ Evasion\ \%})^{0.9}$$

If $(Enemy_{Evasion\ \%} - Player_{Ignore\ Evasion\ \%})$ is lower than 0, it'll be set to 0%.

Launch

The chance of launching the opponent in the air when attacking depends on the *Launch* of the player and the *Ignore Launch* of the enemy as follows:

$$\text{Launch \%} = \text{Player}_{\text{Launch \%}} - \text{Enemy}_{\text{Ignore Launch \%}}$$

If $(\text{Player}_{\text{Launch \%}} - \text{Enemy}_{\text{Ignore Launch \%}})$ is lower than 0, it'll be set to 0%.

Stun

The chance of stunning the enemy when attacking depends on the *Stun* of the player and the *Ignore Stun* of the enemy as follows:

$$\text{Stun \%} = (\text{Player}_{\text{Stun \%}} - \text{Enemy}_{\text{Ignore Stun \%}})^{0.9}$$

If $(\text{Player}_{\text{Stun \%}} - \text{Enemy}_{\text{Ignore Stun \%}})$ is lower than 0, it'll be set to 0%.

If the stun is successful, the stun duration depends on the *Stun RES* and *Control RES* of the enemy as follows:

$$\text{Stun Duration} = \text{Player}_{\text{Stun Time}} \times \left(1 - \frac{\text{Enemy}_{\text{Stun RES}}}{100}\right) \times \left(1 - \frac{\text{Enemy}_{\text{Control RES}}}{100}\right)$$

In PvP, $\text{Player}_{\text{Stun Time}}$ is always 1 and $\text{Enemy}_{\text{Stun RES}}$ is always 60%, so the formula can be written as:

$$\text{Stun Duration}_{\text{PVP}} = 0.4 \times \left(1 - \frac{\text{Enemy}_{\text{Control RES}}}{100}\right)$$

$\text{Enemy}_{\text{Control RES}}$ is the control duration reduction of the enemy:

- If the enemy has no control duration reduction, it'll be 0 so the default stun duration is 0.4 sec.
- If the enemy has some control duration reduction, it'll be greater than zero so the stun duration will be reduced.

For example, if the enemy has the blue ox mount equipped: $\text{Enemy}_{\text{Control RES}} = 30\% \rightarrow \text{Stun Duration}_{\text{PVP}} = 0.4 \times \left(1 - \frac{30\%}{100}\right) = 0.28 \text{ sec}$

Control duration can also be reduced by using the Sunshine Bringer avian or by using an avian with *Debuff Immunity* affix.

NOTE: contrary to what many believe, the *No Speeding* affix does not increase the *Control RES* of the player. So, *No Speeding* does not decrease the duration of stun and disarm. Only *Debuff Immunity* does it.

In PvE, $\text{Enemy}_{\text{Stun RES}}$ might be different than 60%.

Pierce

Every time the player deals damage, there is a 30% chance of piercing the enemy, ignoring 0.01% of the DMG-specific RES (i.e. *Combo*, *Counter*, *Basic Attack*, *Skill* or *Boss DMG RES*, depending on the dealt damage) every 1 pierce unit. *Pal DMG RES* is excluded since it's taken into account in *Pal Inspire*.

If piercing succeeds (30% chance), the final pierce depends on the *Pierce* of the player and the *Ignore Pierce* of the enemy as follows:

$$\text{Pierce} = \text{Player}_{\text{Pierce}} - \text{Enemy}_{\text{Pierce}}$$

If $(\text{Player}_{\text{Pierce}} - \text{Enemy}_{\text{Pierce}})$ is lower than 0, it'll be set to 0.

Block

Every time the player takes damage, there is a 30% chance of blocking the enemy, gaining 0.01% of the DMG-specific RES (i.e. *Combo*, *Counter*, *Basic Attack*, *Skill* or *Boss DMG RES*, depending on the taken damage) every 1 block unit. *Pal DMG RES* is excluded since it's taken into account in *Pal RES*.

If blocking succeeds (30% chance), the final block depends on the *Block* of the player and the *Ignore Block* of the enemy as follows:

$$\text{Block} = \text{Player}_{\text{Block}} - \text{Enemy}_{\text{Block}}$$

If $(\text{Player}_{\text{Block}} - \text{Enemy}_{\text{Block}})$ is lower than 0, it'll be set to 0.

Pal Inspire

Every time the player deals damage, there is a 30% chance of ignoring 0.01% *Pal DMG RES* every 1 pal inspire unit.

If pal inspire succeeds (30% chance), the final pal inspire depends on the *Pal Inspire* of the player and the *Ignore Pal Inspire* of the enemy as follows:

$$\text{Pal Inspire} = \text{Player}_{\text{Pal Inspire}} - \text{Enemy}_{\text{Pal Inspire}}$$

If $(\text{Player}_{\text{Pal Inspire}} - \text{Enemy}_{\text{Pal Inspire}})$ is lower than 0, it'll be set to 0.

Pal RES

Every time the player deals damage, there is a 30% chance of gaining 0.01% *Pal DMG RES* every 1 pal RES unit.

If pal RES succeeds (30% chance), the final pal RES depends on the *Pal RES* of the player and the *Ignore Pal RES* of the enemy as follows:

$$\text{Pal RES} = \text{Player}_{\text{Pal RES}} - \text{Enemy}_{\text{Pal RES}}$$

If $(\text{Player}_{\text{Pal RES}} - \text{Enemy}_{\text{Pal RES}})$ is lower than 0, it'll be set to 0.

Regeneration

Every second, the player regenerates a certain percentage of maximum HP depending on the *Regeneration* of the Player and the *Ignore Regeneration* of the enemy as follows:

$$\text{Regeneration \%} = \text{Player}_{\text{Regeneration \%}} - \text{Enemy}_{\text{Ignore Regeneration \%}}$$

If $(\text{Player}_{\text{Regeneration \%}} - \text{Enemy}_{\text{Ignore Regeneration \%}})$ is lower than 0, it'll be set to 0%.

In PvP, only 30% of player *Regeneration* is taken into account.

Pals

Pals work similar to players, but the pal attributes are used in the formulas instead of the player ones.

Basic Attack

Every time a pal attacks, it performs a pal basic attack.

The damage of a pal basic attack depends on the *DMG Multiplier* of the attacking pal, the *Pal DMG* of the player and the *Pal DMG RES* and *DMG RES* of the enemy as follows:

$$Pal\ Basic\ ATK\ DMG = (Player_{ATK} - Enemy_{DEF}) \times Pal_{DMG\ MULT} \times \frac{Player_{Pal\ DMG}}{100} \times \left(1 - \frac{Enemy_{Pal\ DMG\ RES}}{100}\right) \times \left(1 - \frac{Enemy_{DMG\ RES}}{100}\right)$$

Pals inherit the attack from the player, so the player *ATK* is used in the formula.

Combo

A pal combo is triggered by a pal basic attack and its damage is calculated separately.

The chance of performing a pal combo depends on the *Combo* of the attacking pal and the *Ignore Combo* of the enemy as follows:

$$Pal\ Combo\ \% = Pal_{Combo\ \%} - Enemy_{Ignore\ Combo\ \%}$$

The damage of a pal combo is the pal basic attack damage multiplied by the *Combo Multiplier* of the attacking pal:

$$Pal\ Combo\ DMG = Pal\ Basic\ ATK\ DMG \times \frac{Pal_{Combo\ MULT}}{100}$$

Critical Damage

Pal basic attack and combo can be critical.

The chance of performing a critical attack depends on the *Crit* of the attacking pal and the *Ignore Crit* of the enemy as follows:

$$Pal\ Crit\ \% = Pal_{Crit\ \%} - Enemy_{Ignore\ Crit\ \%}$$

In case of a critical attack, the damage will be multiplied by the ratio between the *Crit DMG* of the attacking pal and *Crit RES* of the enemy as follows:

$$Pal\ Crit\ DMG = Pal\ DMG \times \frac{Pal_{Crit\ DMG}}{Enemy_{Crit\ RES}}$$

Evasion

When a pal attacks, the attack can be evaded by enemy depending on the *Evasion* of the enemy and the *Ignore Evasion* of the attacking pal as follows:

$$Evasion\ \% = (Enemy_{Evasion\ \%} - Pal_{Ignore\ Evasion\ \%})^{0.9}$$

If $(Enemy_{Evasion\ \%} - Pal_{Ignore\ Evasion\ \%})$ is lower than 0, it'll be set to 0%.

Launch

The chance of launching the opponent in the air depends on the *Launch* of the attacking pal and the *Ignore Launch* of the enemy as follows:

$$Launch\ \% = Pal_{Launch\ \%} - Enemy_{Ignore\ Launch\ \%}$$

If $(Pal_{Launch\ \%} - Enemy_{Ignore\ Launch\ \%})$ is lower than 0, it'll be set to 0%.

The only pal that is able to launch the opponent is currently the Benny ($Pal_{Launch\ \%} = 30\%$).

Pal Attributes

All the variables used in the formulas ($Pal_{Combo\ \%}$, $Pal_{DMG\ MULT}$, $Pal_{Combo\ MULT}$, $Pal_{Crit\ DMG}$, $Pal_{Ignore\ Evasion\ \%}$) depends on the pal that is currently attacking and can be found in the info of each pal:



$Pal_{Launch\ \%}$ is not written in the pal info but only in the effect of the Benny.

Skills

Skills are not only the player active skills, but also skills of avians, artifacts, mounts and back accessors.

Skill Damage

A skill damage depends on the damage of the specific skill, the $Skill_{DMG}$ of the player and $Skill_{DMG RES}$ and $DMG RES$ of the enemy as follows:

$$Skill_{DMG} = (Player_{ATK} - Enemy_{DEF}) \times \frac{Skill_{DMG}}{100} \times \frac{Player_{Skill_{DMG}}}{100} \times \left(1 - \frac{Enemy_{Skill_{DMG RES}}}{100}\right) \times \left(1 - \frac{Enemy_{DMG RES}}{100}\right)$$

In case anyone has doubts whether $Enemy_{DMG RES}$ is considered or not for skill damage, it's actually present in the code (see Nerds Corner) and I also debugged step-by-step to verify it.

$Skill_{DMG}$ is the damage written in the skill info for active skills. For avians or items is the one indicated as "Skill DMG" in the info. For example:

- For level 1 Hundred Slashes: $Skill_{DMG} = 4663\%$
- For the bow artifact (less than 5 stars): $Skill_{DMG} = 750\%$
- For a level 1 Dense Cloud avian: $Skill_{DMG} = 1400\%$



Skill Critical Damage

A skill damage can be critical depending on the $Skill_{Crit}$ of the player. In this case, the skill damage is multiplied by the $Skill_{Crit DMG}$ of the player as follows:

$$Skill_{Crit DMG} = Skill_{DMG} \times \left(1 + \frac{Player_{Skill_{Crit DMG}}}{100}\right)^{0.98}$$

Skill critical damage is not reduced by the $Enemy_{Crit RES}$.

Skills with Other Damage Types

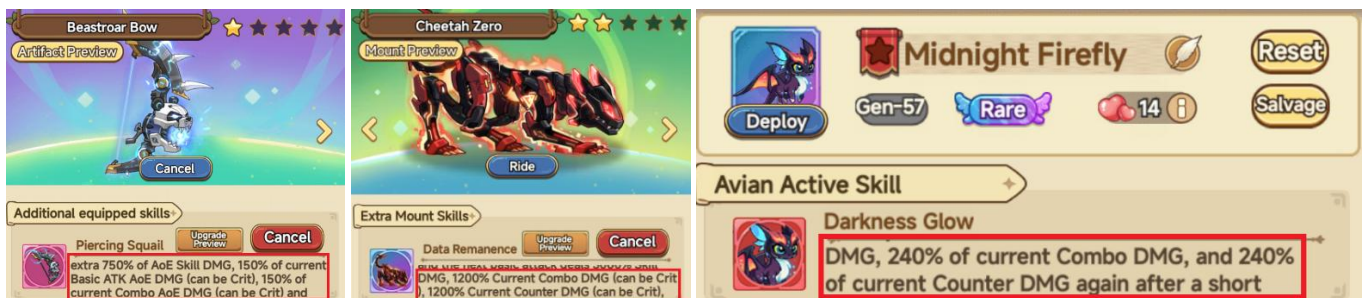
Some skills deal other types of damage in addition to skill damage. For example, the skill of the bow artifact also deals combo, counter and basic attack damage. In this case, each damage is computed using its related formula (combo damage formula for combo, counter damage formula for counter, and so on) multiplied by the skill damage:

$$Final_{DMG} = DMG \times \frac{Skill_{DMG}}{100}$$

$Skill_{DMG}$ is the damage written in the skill info. For example:

- For the bow artifact (less than 5 stars): $Skill_{DMG} = 150\%$ for combo, counter and basic attack
- For the cheetah mount (less than 5 stars): $Skill_{DMG} = 1200\%$ for combo and counter
- For a level 1 Midnight Firefly avian: $Skill_{DMG} = 240\%$ for combo, counter and basic attack

If the damage can crit, its chance depends on the $Crit Rate$ of the player and the $\frac{Player_{Crit DMG}}{Enemy_{Crit RES}}$ ratio will be used as usual to calculate the critical damage.



Skills with Bleed Damage

Some skills may inflict bleed damage, which periodically deals damage every certain amount of time.

In this case, the formulas are the same ones listed before depending on the skill damage type.

Example of bleed damage:



Boss

If the enemy is a boss, the damage (whatever it is: basic attack, combo, skill, etc...) is multiplied by the *Boss DMG* of the player as follows:

$$Final\ DMG = DMG \times \left(1 + \frac{Player_{Boss\ DMG}}{100}\right)$$

If the player is hit by a boss, the damage is reduced by the *Boss DMG RES* of the player as follows:

$$Final\ DMG = DMG \times \left(1 - \frac{Player_{Boss\ DMG\ RES}}{100}\right)$$

Sailing Event

During the sailing event, player stats are replaced by captain stats, which are calculated based on ship and player stats.

Captain stats can be found in the *Maritime Academy*. Here an example:

Captain Battle Attributes	
Final Attributes(Same ship)	
Final HP	+209.1M
Final ATK	+2663.5K
Advanced Attributes	
Crit Rate	+77.18%
Combo	+0%
Counterstrike	+108.4%
Stun	+4.42%
Evasion	+52.26%

The damage formulas are always the same, but captain stats are used instead of player stats.

Cannons

The ship has cannons that attack the enemy. Cannons have the following stats:

- *Cannon Combo*
- *Cannon DMG*
- *Cannon DMG RES*

Cannon stats are increased by equipping ship gears, in particular:

- Sails: increase *Cannon DMG RES*
- Main cannon: increase *Cannon Combo*
- Ram: increase *Cannon DMG*
- Oars: increase *Cannon Combo*
- Wheel: increase *Cannon DMG*

Some examples:

[Epic] Generator Lv.15		[Legendary] Precision Lv.17		[Epic] Standard Sail Lv.15	
Ship HP	676290	Ship ATK	7482	Ship HP	664219
Ship DEF	1883	Ship HP	987936	Ship DEF	1785
Cannon Combo	8.99%	Cannon DMG	14.29%	Cannon DMG RES	8.65%

The final cannon stats are shown in the ship overview, for example:

Flagship Basic Effect	
Level	17 » 18
Ship HP	+2618.0K » +2772.0K
Ship ATK	+18700 » +19800
Ship DEF	+6234 » +6600
Salvaging equipment not in use grants Flagship EXP	
17	3610/48280
Basic Attributes	
Ship ATK	+20473
Ship HP	+2328.4K
Ship DEF	+5604
Bonus Effect	
Cannon DMG RES	+8.65%
Cannon Combo	+19.76%
Cannon DMG	+27.71%

The equipped hull also increases *Cannon DMG*. Moreover, some hulls also have subcannons, which work the same of cannons but with a different damage. For example:



Cannon Damage (Draft)

The final ship cannon damage is calculated using the following (a bit strange) formula (not debugged):

$$\text{Final Ship Cannon DMG} = \frac{\text{Player}_{\text{Captain Basic ATK DMG}}}{100} \times \left(1 + \frac{\text{Player}_{\text{Captain Basic ATK DMG RES}}}{100}\right) \times \left(1 + \frac{\text{Ship Cannon DMG}}{100} + \frac{\text{Hull Cannon DMG}}{100}\right)$$

Don't ask me why the *Captain Basic ATK DMG RES* increases the damage, but that's the way it is.

For example, in my case I had:

- *Ship Cannon DMG* = 34.85%
- *Hull Cannon DMG* = 70%
- *Captain Basic ATK DMG* = 1956.51%
- *Captain Basic ATK DMG RES* = 6.255%

$$\text{Final Ship Cannon DMG} = \frac{1956.51\%}{100} \times \left(1 + \frac{6.255\%}{100}\right) \times \left(1 + \frac{34.85\%}{100} + \frac{70\%}{100}\right) = 42.58$$

Cannon Basic Attack

Every time a cannon attacks, it performs a cannon basic attack.

The damage of a cannon basic attack is calculated as follows:

$$\text{Cannon Basic ATK DMG} = (\text{Player}_{\text{Captain ATK}} - \text{Enemy}_{\text{Captain DEF}}) \times \text{Player}_{\text{Final Ship Cannon DMG}} \times \left(1 - \frac{\text{Enemy}_{\text{Ship Cannon DMG RES}}}{100}\right) \times \left(1 - \frac{\text{Enemy}_{\text{Captain DMG RES}}}{100}\right)$$

The *Crit DMG* of cannons is set to 0%, so it cannot be critical.

A subcannon uses the same formula, but with the *Subcannon DMG* instead of the *Cannon DMG*.

Cannon Combo

A cannon combo is triggered by a cannon basic attack and its damage is calculated separately.

The chance of performing a cannon combo depends on the *Cannon Combo* of the ship and the *Ignore Combo* of the enemy captain as follows:

$$\text{Cannon Combo \%} = \text{Player}_{\text{Ship Cannon Combo \%}} - \text{Enemy}_{\text{Captain Ignore Combo \%}}$$

The damage of a cannon combo is calculated as follows:

$$\text{Cannon Combo DMG} = \text{Cannon Basic ATK DMG} \times \frac{\text{Player}_{\text{Cannon Combo DMG}}}{100} \times \left(1 - \frac{\text{Enemy}_{\text{Captain Combo DMG RES}}}{100}\right)$$

For what I observed, *Player_{Cannon Combo DMG}* is always set to 100%, so it doesn't increase the damage. Therefore, *Cannon Combo DMG* is actually lower than *Cannon Basic ATK DMG*, since it's the same damage but reduced by *Enemy_{Captain Combo DMG RES}*. Basically, it's just an additional damage on top of the basic cannon damage.

HP-Based Damage

HP-Based damage is a special type of damage dealt by avians, items, pals or active skills. Here are some examples:



In principle, the damage is simply a percentage of the player or enemy HP. That's the idea. However, in the algorithm a minimum and maximum damage are also calculated using the *Basic ATK* of the player in order to limit the final damage.

HP-Based damage calculation is more complicated than other damage types and needs multiple steps:

1. The HP damage is calculated as a percentage of the target HP:

$$HP\ DMG = Target\ HP \times \frac{HP_{Perc}}{100}$$

The HP_{Perc} depends on the specific HP-based damage (for example: 2.5% for a level 1 bee avian).

The *Target HP* can be the maximum or current HP of the player or the enemy, depending on the specific skill. For example: for the bee avian it's the maximum HP of the player, for Multiple Shot it's the current HP of the enemy, for Punch Counter it's the current HP of the player.

2. The HP damage is multiplied by the PvP Damage Reduction Factor (see PvP Damage Reduction):

$$HP\ DMG = HP\ DMG \times PvP\ DMG\ Reduction\ Factor$$

3. The basic attack damage of the player or the pal, depending on who is attacking, is calculated using the normal formula, but without considering resistances and crit:

$$\text{If player is attacking: } Basic\ ATK\ DMG = (Player_{ATK} - Enemy_{DEF}) \times \frac{Player_{Basic\ ATK\ MULT}}{100}$$

$$\text{If a pal is attacking: } Basic\ ATK\ DMG = (Player_{ATK} - Enemy_{DEF}) \times Pal_{DMG\ MULT} \times \frac{Player_{Pal\ DMG}}{100}$$

The second formula is used for:

- The Hellflame Feather pal attacks.
- Turtle, lizard and dragon pal attacks if the Dragonweave Circlet is equipped.
- Any pal critical attack (25% chance) if an avian with *Divine Touch* is equipped.

All the rest uses the first formula.

4. The basic attack damage is used to calculate the minimum and the maximum damage as follows:

$$Minimum\ DMG = Basic\ ATK\ DMG \times 0.8$$

$$\text{If damage is based on current HP: } Maximum\ DMG = Basic\ ATK\ DMG \times 50$$

$$\text{If damage is based on maximum HP: } Maximum\ DMG = Basic\ ATK\ DMG \times 100$$

5. The initial HP damage is compared to the minimum and maximum damage:

- If $HP\ DMG \leq Minimum\ DMG$ → $DMG = Minimum\ DMG$
- If $HP\ DMG \geq Maximum\ DMG$ → $DMG = Maximum\ DMG$
- If $Minimum\ DMG < HP\ DMG < Maximum\ DMG$ → $DMG = HP\ DMG$

6. As any other damage type, the calculated damage will be finally divided by the PvP Damage Reduction Factor:

$$Final\ DMG = \frac{DMG}{PvP\ DMG\ Reduction\ Factor}$$

Notes:

- No resistance is taken into account when calculating HP-based damage, so it cannot be reduced by resistances. $Enemy_{DEF}$ is the only thing that can lower the damage in case $HP\ DMG$ is higher than $Maximum\ DMG$ or lower than $Minimum\ DMG$.
- HP-based damage cannot be critical.

Example 1 (Martial Sage with bee avian)

Player:

- HP: 1500B
- ATK: 5000M
- Level: 185
- Basic ATK Multiplier: 5000%
- Using bee avian: deals damage equal to 5% of player maximum HP

Enemy:

- DEF: 2000M
- Level: 180

Calculations:

1. $PvP\ DMG\ Reduction\ Level = \frac{185+180}{2} = 182 \rightarrow PvP\ DMG\ Reduction\ Factor = 485.10$
2. $HP\ DMG = 1500B \times \frac{5\%}{100} \times 485.10 = 36382.5B$
3. $Basic\ ATK\ DMG = (5000M - 2000M) \times \frac{5000\%}{100} = 150B$
4. $Minimum\ DMG = 150B \times 0.8 = 120B$
 $Maximum\ DMG = 150B \times 100 = 15000B$
5. Since $HP\ DMG \geq Maximum\ DMG \rightarrow DMG = 15000B$
6. $Final\ DMG = \frac{15000B}{485.10} = 30.92B$ (2.06% of maximum HP, since the damage was limited)

Example 2 (Plume Monarch with Multiple Shots on avian)

Player:

- ATK: 8000M
- Level: 180
- Basic ATK Multiplier: 4000%
- Using avian with multiple shots: deals damage equal to 1% of enemy current HP

Enemy:

- HP: 1500B
- Current HP: 1300B (let's suppose it took some damage)
- DEF: 2000M
- Level: 185

Calculations:

1. $PvP\ DMG\ Reduction\ Level = \frac{185+180}{2} = 182 \rightarrow PvP\ DMG\ Reduction\ Factor = 485.10$
2. $HP\ DMG = 1300B \times \frac{1\%}{100} \times 485.10 = 6306.3B$
3. $Basic\ ATK\ DMG = (8000M - 2000M) \times \frac{4000\%}{100} = 240B$
4. $Minimum\ DMG = 240B \times 0.8 = 192B$
 $Maximum\ DMG = 240B \times 50 = 12000B$
5. Since $Minimum\ DMG < HP\ DMG < Maximum\ DMG \rightarrow DMG = 6306.3B$
6. $Final\ DMG = \frac{6306.3B}{485.10} = 13B$ (1% of current HP, since the damage was not limited)

Conclusions

- If a player has a lot of HP and uses HP-damage based on a high percentage of its maximum HP (for example: avian bee, pagoda artifact), it's very likely that the *HP DMG* will be higher than the *Maximum DMG*.
Therefore, it makes sense to increase the *ATK* and the *Basic ATK Multiplier* to increase the final damage, since it'll increase the *Maximum DMG*.
- If a player uses HP-damage based on a low percentage of current HP (for example: Multiple Shots), it's very likely that the *HP DMG* will be between the *Minimum DMG* and the *Maximum DMG*.
Therefore, increasing the *Basic ATK Multiplier* won't help much, because the damage will be the same most of the times unless it's very close to *Maximum DMG* (unlikely to happen if the class has a high *ATK* like archers). However, it will increase the *Minimum DMG*, which deals a bit more damage when the enemy is almost dead (very low *HP*).
- In PvE, HP-based damage works, but it isn't very effective. Since monsters have enormous amounts of HP (often in the millions or thousands of billions), the *HP DMG* will always exceed the *Maximum DMG*, meaning it will always be capped at that limit (i.e. 50 or 100 times the basic *ATK* damage). Considering that monsters have low resistances and Crit RES, a critical combo or counter will deal more damage than HP-based damage.

PvP Damage Reduction

In PvP, the final damage is divided by a factor that depends on the average levels of the two fighters rounded down:

$$PvP \text{ Damage Reduction Level} = \frac{Player_{Level} + Enemy_{Level}}{2}$$

$$Final \text{ DMG} = \frac{DMG}{PvP \text{ Damage Reduction Factor}}$$

This factor increases with the average level of the two fighters. Therefore, if a high-level player fights a low-level player, it'll deal more damage since the factor will be lower than when fighting a high-level player.

In PvE damage is not reduced, so the factor is always set to 1.

The *PvP Damage Reduction Factors* for every level are listed in the following table:

Level	PvP Damage Reduction Factor Last Update: 04/2025
1	1.00
2	1.00
3	1.00
4	1.00
5	1.00
6	1.00
7	1.00
8	1.10
9	1.15
10	1.40
11	1.60
12	1.70
13	2.00
14	2.10
15	2.40
16	2.50
17	2.55
18	2.95
19	3.05
20	3.45
21	3.85
22	4.00
23	4.15
24	4.25
25	4.75
26	4.90
27	5.05
28	5.15
29	5.30
30	5.80
31	6.45
32	6.60
33	6.75
34	6.95
35	7.65
36	7.80
37	7.95
38	8.15
39	8.35
40	8.50
41	9.35
42	9.50
43	9.70
44	9.90
45	10.05
46	10.25
47	10.45
48	10.65
49	10.80
50	11.00
51	11.95
52	12.15
53	12.35
54	12.55
55	12.75
56	12.95
57	13.15
58	13.35

59	13.55
60	13.75
61	14.90
62	15.10
63	15.35
64	15.55
65	16.10
66	16.70
67	17.30
68	17.90
69	18.60
70	19.20
71	19.90
72	20.70
73	21.40
74	22.20
75	23.00
76	23.90
77	24.80
78	25.70
79	26.60
80	27.60
81	28.60
82	29.70
83	30.80
84	31.90
85	33.10
86	34.30
87	35.60
88	36.90
89	38.20
90	39.60
91	41.10
92	42.60
93	44.20
94	45.80
95	47.50
96	49.20
97	51.00
98	52.90
99	54.90
100	56.90
101	59.00
102	61.20
103	63.40
104	65.80
105	68.20
106	70.70
107	73.30
108	76.00
109	78.80
110	81.70
111	84.70
112	87.80
113	91.10
114	94.40
115	97.90
116	101.50
117	105.20
118	109.10
119	113.10
120	117.30
121	121.60
122	126.10
123	130.80
124	135.60
125	140.60
126	145.80
127	151.00
128	156.20
129	161.40
130	166.60
131	172.00
132	177.40

133	182.80
134	188.20
135	193.60
136	199.20
137	204.80
138	210.40
139	216.00
140	221.60
141	227.40
142	233.20
143	239.00
144	244.80
145	250.60
146	256.60
147	262.60
148	268.60
149	274.60
150	280.60
151	286.70
152	292.80
153	298.90
154	305.00
155	311.20
156	317.40
157	323.60
158	329.80
159	336.00
160	342.30
161	348.60
162	354.90
163	361.20
164	367.50
165	373.90
166	380.30
167	386.70
168	393.10
169	399.50
170	406.00
171	412.50
172	419.00
173	425.50
174	432.00
175	438.60
176	445.20
177	451.80
178	458.40
179	465.00
180	471.70
181	478.40
182	485.10
183	491.80
184	498.50
185	505.30
186	512.10
187	518.90
188	525.70
189	532.50
190	539.40
191	546.30
192	553.20
193	560.10
194	567.00
195	574.00
196	581.00
197	588.00
198	595.00
199	602.00
200	609.00
201	616.10
202	623.20
203	630.30
204	637.40
205	644.50
206	651.70

207	658.90
208	666.10
209	673.30
210	680.50
211	687.80
212	695.10
213	702.40
214	709.70
215	717.00
216	724.40
217	731.80
218	739.20
219	746.60
220	754.00

Attributes Maximum Limits

Some attributes may have a maximum cap. This doesn't mean you can't exceed this cap with your raw stats (for example, it's easy to go over 100% combo or counter), but it is the highest value considered after all buffs and debuffs are applied.

For instance, if you have 90% combo DMG RES, it will be capped at 80%. However, if your opponent is an archer and its class skill that reduces the resistance by 40% is active, the final combo DMG RES will drop to 50% due to the debuff and the cap won't apply.

Please note that this doesn't apply for ignore combo, ignore counter, etc... because they are not debuffs, but simply stats. This means that combo%, counter%, etc... are capped to 100% before the "ignore" stat is subtracted. For example: if a player has 120% counter and the opponent has 40% ignore counter, the final counter will be 60%, and not 80%.

All attributes not listed here have no maximum cap.

Attributes	Maximum Value
Crit Rate	100%
Combo	100%
Counter	100%
Stun	100%
Ignore Stun	100%
Launch	100%
Ignore Launch	100%
Control Duration Reduction	100%
Combo DMG RES	80%
Counter DMG RES	80%
Skill DMG RES	80%
Pal DMG RES	80%
Basic ATK DMG RES	80%
Boss DMG RES	80%
Cannon DMG RES	60%
DMG RES	80%

Nerds Corner

Source code from which the formulas were taken.

Damage Resistance

```
var o = t("calHurt", (function(t, a, r) {
  var d = a.data.getAttrib(e.resist)
  , i = a.data.getAttrib(e.pve_resist)
  , o = r.data.getAttrib(e.pve_dam);
  t = n.roundInt(t * n.round(1 + o));
  var u = n.roundInt(n.roundInt(t * n.round(1 - d)) * n.round(1 - i));
  return Math.max(1, u)
})
))
```

pve_resist and pve_dam are 0

Basic Attack

```
t("normalHurt", (function(t, r, a) {
  var d = t.data.getAttrib(e.att)
  , o = r.data.getAttrib(e.def)
  , m = r.data.getAttrib(e.def_coe)
  , p = t.data.getAttrib(e.crit_dam)
  , _ = Math.max(.5, r.data.getAttrib(e.crit_def))
  , s = r.data.getAttrib(e.att_resist)
  , l = t.data.getAttrib(e.att_dam);
  if (t.config.type == i.Partner) {
    d = t.parent.data.getAttrib(e.att);
    s = r.data.getAttrib(e.partner_resist);
    l = t.data.getAttrib(e.partner_dam);
    var c = t.parent.data.getAttrib(e.partner_dam_extra);
    l = n.round(1 * c);
  } else
    t.config.type == i.Gun ? (l = t.data.getAttrib(e.partner_dam),
    s = r.data.getAttrib(e.season_cannon_att_def),
    s = b(r, t, s, e.season_cannon_att_def)) : s = u(r, t, s, e.att_resist);
  var A = n.roundInt(Math.max(n.roundInt(d - o * (1 + m)), 1) * n.round(1 * n.round(1 - s)));
  return A = g(A, r, t);
  1 == a || (A = n.roundInt(A * Math.max(1.5, n.round(p / _))),
  Math.max(1, A)
})
))
```

Pal Inspire/RES applied here (calSuppressAndInspire)

Pal

Pierce/Block applied here (calArmorAndBlock)

Cannon

DMG RES applied here (calHurt)

Combo

Combo rate (for both Combo and Pal Combo):

```
t("checkDoubleAct", (function(t, a) {
  var r = t.data.getAttrib(e.double_hit)
  , d = 0;
  a && (d = a.data.getAttrib(e.ignore_double_hit));
  var i = n.roundInt(1e4 * Math.max(n.round(r - d), 0));
  if (i <= 0)
    return !1;
  return !1;
  var o = t.battleMain.random.randomInt(0, 1e4);
  return t.battleMain.printLogFlag && t.battleMain.printLogDebug("连击 rand: " + o + " tem: " + i),
  o <= i
})
))
```

Combo damage:

```
, m = (t("normalDoubleHurt", (function(t, r, a) {
  var d = t.data.getAttrib(e.att)
  , o = r.data.getAttrib(e.def)
  , m = r.data.getAttrib(e.def_coe)
  , p = t.data.getAttrib(e.crit_dam)
  , _ = Math.max(.5, r.data.getAttrib(e.crit_def))
  , s = r.data.getAttrib(e.double_hit_def)
  , l = u(r, t, s, e.double_hit_def)
  , c = t.data.getAttrib(e.double_hit_dam)
  , A = 0;
  if (t.config.type == i.Partner) {
    d = t.parent.data.getAttrib(e.att);
    var f = t.data.getAttrib(e.partner_dam)
    , h = t.parent.data.getAttrib(e.partner_dam_extra);
    f = n.round(f * h);
    var M = r.data.getAttrib(e.partner_resist)
    , v = b(r, t.parent, M, e.partner_resist)
    , I = n.roundInt(Math.max(n.roundInt(d - o * (1 + m)), 1) * f) * n.round(1 - v);
    A = n.roundInt(n.roundInt(I) * c);
    A = n.roundInt(A);
  } else if (t.config.type == i.Gun) {
    var x = t.data.getAttrib(e.partner_dam);
    x = t.data.getAttrib(e.partner_dam);
    var k = r.data.getAttrib(e.season_cannon_att_def)
    , L = n.roundInt(Math.max(n.roundInt(d - o * (1 + m)), 1) * x) * n.round(1 - k);
    A = n.roundInt(n.roundInt(L) * c) * n.round(1 - l);
    A = n.roundInt(A);
  } else
    A = n.roundInt(Math.max(n.roundInt(d - o * (1 + m)), 1) * c) * n.round(1 - l);
    A = n.roundInt(A);
  return A = g(A, r, t);
  1 == a || (A = n.roundInt(A * Math.max(1.5, n.round(p / _))),
  Math.max(1, A)
})
))
```

Pierce/Block applied here (calArmorAndBlock)

Pal

Cannon

DMG RES applied here (calHurt)

Counter

Counter rate:

```
t("checkCounterAct", (function(t, a) {
    var r = t.data.getAttrib(e.counter)
    , d = a.data.getAttrib(e.ignore_counter)
    , i = n.roundInt(1e4 * Math.max(n.round(r - d), 0));
    if (i <= 0)
        return !1;
    var o = t.battleMain.random.randomInt(0, 1e4);
    return t.battleMain.printLogFlag && t.battleMain.printLogDebug("反击 rand: " + o + " tem: " + i),
    o <= i
}),
)),
```

Counter damage:

```
t("normalCounterHurt", (function(t, r, a) {
    var i = t.data.getAttrib(e.att)
    , d = r.data.getAttrib(e.def)
    , o = r.data.getAttrib(e.def_coe)
    , b = t.data.getAttrib(e.crit_dam)
    , m = Math.max(.5, r.data.getAttrib(e.crit_def))
    , p = r.data.getAttrib(e.counter_def)
    , _ = u(r, t, p, e.counter_def)
    , s = t.data.getAttrib(e.counter_dam)
    , l = n.roundInt(Math.max(n.roundInt(i - d * (1 + o)), 1) * s) * n.round(1 - _);
    return l = n.roundInt(1),
    l = g(l, r, t),
    l == a || (l = n.roundInt(1 * Math.max(1.5, n.round(b / m))),
    Math.max(1, l)
}),
```

Pierce/Block applied here (calArmorAndBlock)

DMG RES applied here (calHurt)

Skills

```
y = n.round(y * r.data.getSkillFactAttrValue(this.skillPar, this.runner.useSkill.config.id, u,
active_skillDamagePar));
var M = 0
, B = r.buffCtr.getBuffByType(o.SKILL_DAMAGE_ADD);
if (B.length > 0)
    for (var C, x = a(B); !C = x().done; ) {
        var E = C.value;
        M = n.roundInt(M + E.calDamage(y, t, this.runner.useSkill.config.id))
    }
m(B);
var R = this.ignoreFlag & b.T1045 ? 1 : r.data.getAttrib(u.skill_dam_extra)
, D = n.roundInt(y * R)
, P = s.Hurt;
if (!(this.ignoreFlag & b.SkillCrit) && p(r)) {
    var L = r.data.getAttrib(u.skill_crit_dam);
    D = n.roundInt(D * n.round(1 + L));
    D = n.roundInt(Math.pow(D, .98));
    P = s.Hurt_Crit
}
if (this.ignoreFlag & b.UseCrit && (r, t, !0) == g.Crit) {
    var F = r.data.getAttrib(u.crit_dam)
    , w = Math.max(.5, t.data.getAttrib(u.crit_def));
    D = n.roundInt(D * Math.max(1.5, n.round(F / w)));
    P = s.Hurt_Crit
}
var U = r.data.getAttrib(u.boss_dam);
this.owner.config.type == d.Boss && U > 0 && (D = n.roundInt(D * n.round(1 + U))),
D = n.roundInt(D * M);
var G = r.skillctr.getRecordDamage(this.runner.useSkill.config.id);
D = n.roundInt(D * n.round(1 + n.round(G / 1e4))),
D = n.roundInt(D * this.runner.useSkill.counterDamage);
var N = this.calResistPar();
D = v(n.roundInt(D * N), t, r);
```

Skill Damage

"y" is the damage calculated with usual formula: (ATK-DEF) x DMG_MULT etc...

Player Skill DMG

Skill Crit

For skills that deal combo, counter, etc... damages and can crit

Specific DMG RES applied here

DMG RES applied here (calHurt)

```
l.calResistPar = function() {
    var t = this.owner
    , r = this.runner.cast
    , a = t.data.getAttrib(u.skill_resist)
    , e = p(t, r, a, u.skill_resist)
    , i = n.round(1 - e);
    switch (this.calType) {
        case 4:
        case 5:
            var l = t.data.getAttrib(u.att_resist)
            , s = p(t, r, l, u.att_resist);
            i = n.round(1 - s);
            break;
        case 6:
            var d = t.data.getAttrib(u.double_hit_def)
            , o = p(t, r, d, u.double_hit_def);
            i = n.round(1 - o);
            break;
        case 7:
            var f = t.data.getAttrib(u.counter_def)
            , c = p(t, r, f, u.counter_def);
            i = n.round(1 - c);
            break;
        case 10:
            var g = t.data.getAttrib(u.partner_resist)
            , h = p(t, r, g, u.partner_resist);
            i = n.round(1 - h);
    }
    return i
}
```

```
function(t) {
    p = t.checkSkillCirt,
    _ = t.checkHit,
    v = t.calHurt,
    k = t.skillHpsteal
}
```

Evasion/Ignore Crit

```
t("checkHit", (function(t, r, i) {
    void 0 === i && (i = !1);
    var d = t.data.getAttrib(e.hit)
    , u = i ? 0 : r.data.getAttrib(e.miss)
    , b = a.battle_up_limit
    , g = t.data.getAttrib(e.crit_rate)
    , p = r.data.getAttrib(e.ignore_crit_rate)
    , _ = Math.max(g - p, 0)
    , s = Math.min(Math.max(n.round(u - d), 0), b[0][1])
    , l = n.round(Math.pow(n.round(100 * s), n.round(a.miss_correct / 1e4)) / 100)
    , c = t.battleMain.random.randomInt(0, 1e4);
    m[o.Miss] = n.roundInt(1e4 * 1),
    m[o.Normal] = n.roundInt(m[o.Miss] + n.roundInt(n.round(1 - l) * n.round(1 - _) * 1e4)),
    m[o.Crit] = n.roundInt(m[o.Normal] + n.roundInt(n.round(1 - l) * _ * 1e4));
    var A = -1;
    t.battleMain.printLogFlag && t.battleMain.printLogDebug("rand: " + c + " crit: " + m[o.Crit] + " miss: " + m[o.Miss]);
    for (var f = 0; f < 2; f++)
        if (m[f] > 0 && c <= m[f]) {
            A = f;
            break
        }
    return -1 == A && (A = o.Crit),
    A
}),
```

80%

miss_correct: 9e3

Stun

Check if success:

```
t("checkDizz", (function(t, a) {
    var d = t.data.getAttrib(e.vertigo)
        , i = a.data.getAttrib(e.vertigo_def)
        , o = Math.max(0, n.round(d - i));
    if (o <= 0)
        return !1;
    var u = n.round(Math.pow(n.round(100 * o), n.round(r.vertigo_correct / 1e4)) / 100);
    if ((u = n.roundInt(1e4 * u)) <= 0)
        return !1;
    var g = t.battleMain.random.randomInt(0, 1e4);
    return t.battleMain.printLogFlag && t.battleMain.printLogDebug("眩晕 rand: " + g + " tem: " + u),
    g <= u
})
)),
```

vertigo_correct: 9e3

Stun duration:

```
t(r.cast, t))<———— It calls checkDizz
var St = r.cast.data.getAttrib(1.vertigo_times) * i.round(1 - t.data.getAttrib(1.vertigo_res));
if ((St = i.round(St)) > 0) {
    r.addBuff(t, Ht.vertigo_time, St);
}
if (v.skillPar = r,
"dizz" == u.action && 0 == u.param1 || "ban_act" == u.action) {
    var y = t.data.getAttrib(k.CONTROL_RES);
    n = a.round(n - a.round(n * y)),
}
```

Duration calculation without control reduction

Control duration reduction

Launch

```
t("checkThrowHit", (function(t, a) {
    var r = t.data.getAttrib(e.suspend)
        , d = a.data.getAttrib(e.suspend_def)
        , i = n.roundInt(1e4 * n.round(r - d));
    if (i <= 0)
        return !1;
    var o = t.battleMain.random.randomInt(0, 1e4);
    return t.battleMain.printLogFlag && t.battleMain.printLogDebug("击飞 rand: " + o + " tem: " + i),
    o <= i
})
)),
```

Pierce/Block

```
var u = t("calArmorAndBlock", (function(t, r, a, i) {
    var o = [0, 0, 0]
        , u = r.data.getAttrib(e.armor_penetration)
        , b = t.data.getAttrib(e.ignore_armor_penetration)
        , g = r.data.getAttrib(e.ignore_block)
        , m = t.data.getAttrib(e.block);
    if (u > b) {
        var p = r.data.getAttrib(e.armor_penetration_rate);
        o[0] = n.roundInt(1e4 * p)
    }
    if (m > g) {
        var _ = t.data.getAttrib(e.block_rate);
        o[1] = o[0] + n.roundInt(1e4 * _)
    }
    o[2] = 1e4;
    for (var s = -1, l = r.battleMain.random.randomInt(0, 1e4), c = 0; c < 2; c++)
        if (o[c] > 0 && 1 <= o[c]) {
            s = c;
            break
        }
    -1 == s && (s = 2);
    var A = a;
    0 == s ? (A = n.round(a - Math.min(.5, (u - b) / 1e4)),
    r.hit(r, n.round(1e4 * A), d.Armor, r.data.getAttrib(e.speed))) : 1 == s && (A = n.round(a + Math.min(.5, (m - g) / 1e4)),
    t.hit(t, n.round(1e4 * A), d.Armor_def, t.data.getAttrib(e.speed)));
    var f = t.data.getAttribMeta(i).config;
    if (0 != f.up_limit) {
        var h = f.up_limit;
        2 == f.num_type && (h = n.round(h / 1e4)),
        A = Math.min(A, h)
    }
    return A
})
})
```

Pal Inspire/Pal RES

```
b = t("calSuppressAndInspire", (function(t, r, a, i) {
    var o = [0, 0, 0]
        , u = r.data.getAttrib(e.partner_inspire)
        , b = t.data.getAttrib(e.ignore_partner_inspire)
        , g = r.data.getAttrib(e.ignore_partner_suppress)
        , m = t.data.getAttrib(e.partner_suppress);
    if (u > b) {
        var p = r.data.getAttrib(e.partner_suppress_rate);
        o[0] = n.roundInt(1e4 * p)
    }
    if (m > g) {
        var _ = t.data.getAttrib(e.partner_inspire_rate);
        o[1] = o[0] + n.roundInt(1e4 * _)
    }
    o[2] = 1e4;
    for (var s = -1, l = r.battleMain.random.randomInt(0, 1e4), c = 0; c < 2; c++)
        if (o[c] > 0 && 1 <= o[c]) {
            s = c;
            break
        }
    -1 == s && (s = 2);
    var A = a;
    0 == s ? (A = n.round(a - Math.min(.5, (u - b) / 1e4)),
    r.hit(r, n.round(1e4 * A), d.Inspire, r.data.getAttrib(e.speed))) : 1 == s && (A = n.round(a + Math.min(.5, (m - g) / 1e4)),
    t.hit(t, n.round(1e4 * A), d.Suppress, t.data.getAttrib(e.speed)));
    var f = t.data.getAttribMeta(i).config;
    if (0 != f.up_limit) {
        var h = f.up_limit;
        2 == f.num_type && (h = n.round(h / 1e4)),
        A = Math.min(A, h)
    }
    return A
})
})
```

HP-Based Damage

This is the standard function:

```
i_calHpHurt = function(t, r, e) {
  var i = 3 == this._calType || 2 == this._calType || 8 == this._calType || 9 == this._calType;
  if (i = 1 || 0 == this._calType && this._attribId == u.hp) {
    var l = n.roundInt(e * this.skillPar);
    if (l = n.roundInt(l * t.battleMain.injuryReduce),
    this._limit) {
      var c = r.data.getAttrib(u.att)
      , g = t.data.getAttrib(u.def)
      , h = t.data.getAttrib(u.def_coe)
      , p = r.data.getAttrib(u.att_dam);
      if (r.isCallType && r.config.Type == d.Partner) {
        c = r.parent.data.getAttrib(u.att);
        var _ = r.data.getAttrib(u.partner_dam)
        , v = r.data.getAttrib(u.partner_dam_extra);
        p = n.round(p * _ * v)
      }
      var k = n.roundInt(Math.max(n.roundInt(c - g * (1 + h)), 1) * p);
      T = n.roundInt(k * this._limit[0])
      , b = n.roundInt(k * this._limit[1]);
      l = Math.max(1, T);
      l = Math.min(l, b);
    }
    this.runner.healthTarget(t, l, s.Hurt, !1, this.config.id);
    var y = t.buffCtr.getBuffByType(o.VAMPIRE);
    if (y.length > 0)
      for (var A, I = a(y); !(A = I()).done; ) {
        A.value.calDamage(l, t, this.runner.useSkill.config.id)
      }
    m(y);
    for (var H, S = t.skillctr.skillEffects, M = a(S); !(H = M()).done; ) {
      var B = H.value;
      B.triggerType == f.HP_Hurt && (B.num++,
      B.limit > 0 && B.num % B.limit != 0 || (0 == B.useType ? B.runner.addTask(B.id, t.position, B.runner, t)
      : 1 == B.useType && t.skillctr.addSkill(B.id)))
    }
    return !0
  }
  return !1
}
```

HP DMG

Multiplied by PvP DMG reduction factor

Pal

Basic ATK DMG

Min/Max limits

Final DMG

This is the function used only by Dragonic Resonance skill, which is slightly different because it deals HP-damage based on the enemy *DMG RES*:

```
return e.onBegin = function() {
  var t = this.owner
  , i = this.runner.cast;
  if (!(t.battleMain.battleFlag & 1.NOT_HURT)) {
    var n = t.data.getAttrib(u.hp)
    , e = i.data.getAttrib(u.resist)
    , o = f(t, i, e, u.resist);
    if (n = a.roundInt(n * this.skillPar) + a.roundInt(n * this._resistPar * o),
    n = a.roundInt(n * t.battleMain.injuryReduce),
    this._limit) {
      var d = i.data.getAttrib(u.att)
      , h = t.data.getAttrib(u.def)
      , p = t.data.getAttrib(u.def_coe)
      , g = i.data.getAttrib(u.att_dam)
      , m = a.roundInt(Math.max(a.roundInt(d - h * (1 + p)), 1) * g)
      , _ = a.roundInt(m * this._limit[0])
      , v = a.roundInt(m * this._limit[1]);
      n = Math.max(n, _);
      n = Math.min(n, v);
    }
    this.runner.healthTarget(t, n, s.Hurt, !1, this.config.id);
    for (var b, A = t.skillctr.skillEffects, H = r(A); !(b = H()).done; ) {
      var k = b.value;
      k.triggerType == c.HP_Hurt && (k.num++,
      k.limit > 0 && k.num % k.limit != 0 || this.runner.useSkill.config.id == k.runner.
      useSkill.config.id && this.runner.cast != k.runner.cast || (0 == k.useType ? k.
      runner.addTask(k.id, t.position, k.runner, t) : 1 == k.useType && t.skillctr.
      addSkill(k.id)))
    }
  }
}
```

Pierce/Block applied here
(calArmorAndBlock)

PvP Damage Reduction

```
o.start = function() {
    var t = this.battleMain.mainCtr.player.data.level
    , e = this.arenaPlayerCtr.player.data.level
    , a = n.roundInt((t + e) / 2) ← Average of player levels
    , i = configLevel.getDataByKey(a);
    this.battleMain.injuryReduce = n.round(i.pvp_injury_reduce / 1e4),
    this.battleMain.shieldDecay = n.round(r.shield_correct / 1e4),
    this.battleMain.treatDecay = n.round(r.hp_recovery_correct / 1e4)
}

for (var D, w = 0, L = !1, O = i(t); !(D = O()).done; ) {
    var R = D.value
    , x = R.health;
    switch (R.healthType) {
        case v.Hurt:
        case v.Hurt_Crit:
        case v.Hurt_Ret:
        case v.Hurt_Share_Damage:
        case v.Hurt_Share_Damage_Crit:
        case v.Hurt_Double:
        case v.Hurt_Double_Crit:
        case v.Real_Damage:
        case v.Hurt_Bleed:
        case v.Hurt_Bleed_Crit:
        case v.Hurt_Counter:
        case v.Hurt_Counter_Crit:
            if (this.battleMain.runningToPart)
                continue;
            if (x = Math.max(s.roundInt(x / this.battleMain.injuryReduce), 1),
```

Sailing Event

Cannon damage:

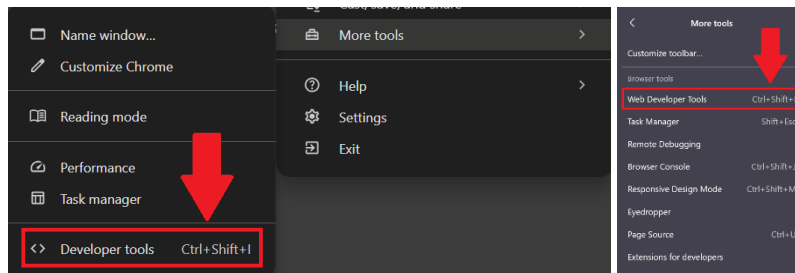
```
l.addgun = function(t, a, n, l, o) {
    void 0 === o && (o = 1);
    for (var l = this.battleMain, p = d[o], c = configSeason_ship_appearance.getDataByKey(a.gunId),
        f = null != a.gun2 ? 2 : 1, h = a.config.cannon_plot, g = e.roundInt(t.data.getAttribByInt(r.att)),
        y = e.roundInt(1e4 * t.data.getAttrib(r.speed)),
        b = t.data.getAttrib(r.att_dam), _ = t.data.getAttrib(r.att_resist), v = e.round(b * (1 + _)), M = 0; M < p.length; M++) {
        var I = p[M]
        , C = c.cannon_type
        , A = l.unitMgr.addUnit(C, n)
        , P = h[I - 1];
        if (2 == o ? (A.direction = -1,
            u.x = e.roundInt(720 - i[0] - P[0]),
            u.y = e.roundInt(s[2] + i[1] + P[1])) : (u.x = e.roundInt(i[0] + P[0]),
            u.y = e.roundInt(s[1] + i[1] + P[1])),
            M >= f)
            A.statectr.banAct++;
        else {
            var m = 0 == M ? a.gun1 : a.gun2;
            A.data.getAttribMeta(r.speed).baseValue = e.roundInt(y + m.speed),
            A.data.getAttribMeta(r.partner_dam).baseValue = e.roundInt(v * e.round(m.dam_add / 1e4) * 1e4),
            A.data.getAttribMeta(r.double_hit).baseValue = m.double_hit
        }
        A.position = u,
        A.data.getAttribMeta(r.att).baseValue = g,
        I < 3 && (A.animatorctr.aniAddName = "up")
    }
}
```

Increased by basic ATK
DMG RES

How to Browse the Code

If you want to browse the code yourself:

1. Open LOM in the browser (<https://lom.joynetgame.com>) and wait for the game to load.
2. Open the *Developer Tools* that are integrated in the browser. If you use Chrome or Firefox, you'll find it under *More Tools* or by pressing CTRL+SHIFT+I.



3. In the *Developer Tools*, open the tab where you can browse the code. The name may change depending on the browser (for example: *Sources* in Chrome, *Debugger* in Firefox).
4. There are lots of scripts, the one that contains everything is located under: *lom.joynetgame.com* → *assets* → *script*. The script name may change after game updates. It's easy to recognize because it has more than half a million lines of code.
5. Have fun. You can read the code and debug it using the browser debugger.
Remember that the script is not the original source code, but it's the output of a code minifier tool, which makes the code easy and fast to load for the browser. Therefore, it's not meant to be human-readable but machine-readable. In other words: it'll be hard to read and understand.